Spherical Convolutional Neural Network

Sam Hawkes

July 2026 - an update from August 2024

Spherical CNNs study how to build a convolutional network for data that lives on the surface of a sphere, rather than on a flat grid.

Think of a photograph pinned onto a globe. An ordinary CNN is built to slide the same filter across a flat photo and get the same answer no matter where on the photo an edge appears. But if the photo is pinned to a globe instead and the globe is spun, a flat CNN has no natural way to guarantee it still recognizes the same features. The question is: how do we build a network that "sees" the same thing regardless of how the sphere has been rotated?

In general, we replace the flat image $I(x, y)$ with a function $f(\theta, \phi)$ living on the sphere's surface, and we replace ordinary translation of the image with rotation of the sphere.

Spherical Coordinates

A point on the sphere is addressed by two angles instead of a pixel coordinate:

- $\theta \in [0, \pi]$, the polar angle, measured from the north pole,
- $\phi \in [0, 2\pi)$, the azimuthal angle, measured around the equator.

A signal on the sphere is a function $f(\theta, \phi)$ giving its value at every such point. To keep things tractable, the signal is only defined on a small patch of the sphere (a bounded window of latitude and longitude) rather than the whole surface — the rest is treated as empty background.

A flat image doesn't naturally live at these angles, so we need a rule for reading off a value at any (θ, ϕ). This is solved with bilinear interpolation: each spherical sample point looks up its four nearest flat-image pixels and blends between them.

Spherical Harmonic Decomposition

Instead of working with the raw values of f directly, we decompose it into a basis of waves on the sphere, called *spherical harmonics* $Y_l^m(\theta, \phi)$:

$$f(\theta, \phi) = \sum_{l=0}^{\infty} \sum_{m=-l}^{l} a_l^m \, Y_l^m(\theta, \phi).$$

This is the direct sphere-analogue of an ordinary Fourier series. Each harmonic is indexed by two numbers:

- degree l — how much spatial detail it captures (like frequency),
- order m — how it varies going around the sphere azimuthally.

The coefficients a_l^m tell us how much of each harmonic is present in f. This always has a solution (any reasonable signal on the sphere can be written this

way), and, just as with Fourier series, lets us work in a frequency domain instead of a spatial domain.

The coefficients a_l^m are computed from the raw signal f using a transform matrix built from the harmonics evaluated on a sampling grid. Since that matrix generally isn't square, its pseudo-inverse is used instead, giving a least-squares solution.

Rotation as a Phase Shift

Instead of thinking about resampling the sphere every time it's rotated, we can think about what rotation does to the coefficients directly.

Suppose the sphere is rotated by an angle α about the polar axis. To avoid the cost of resampling the whole signal, we ask: what happens to each a_l^m instead?

The answer turns out to be extremely simple:

$$a_l^m \longrightarrow e^{im\alpha}\, a_l^m.$$

Rotation about the polar axis is *exactly* a per-coefficient multiplication by a complex phase — nothing moves between coefficients, and no interpolation is needed. This is the sphere's analogue of the shift theorem in ordinary Fourier analysis.

This means that rotation in the spatial domain equals a phase multiplication in the harmonic domain, and this identity is exact rather than approximate.

Rotation-Equivariant Convolution

A convolution on the sphere should have the property that rotating the input and then convolving gives the same result as convolving and then rotating. This is called equivariance, and it's the direct sphere-analogue of the translation-equivariance an ordinary CNN relies on.

We achieve this by learning one weight W_l per degree l, applied to every order m within that degree. Because rotation only rescales each coefficient by a phase $e^{im\alpha}$ without moving it to a different (l, m) slot, and the convolution is linear,

$$W_l\big(e^{im\alpha} a_l^m\big) = e^{im\alpha}\, W_l\big(a_l^m\big).$$

The rotation simply passes through the layer unchanged. This is only possible because the weight is shared across all m within a degree. If every coefficient had its own separate weight, this cancellation would not happen and equivariance would be lost.

Nonlinearity on the Sphere

Exact harmonic-domain processing is cheap, but a purely linear network is limited, so we need a nonlinearity, the sphere's analogue of ReLU. The problem is

that an ordinary ReLU is not defined for complex numbers, and the harmonic coefficients are complex.

We fix this the same way many spectral methods do: temporarily leave the harmonic domain, apply the nonlinearity where it's well-defined, and transform back.

$$\text{coefficients} \xrightarrow{\text{inverse transform}} \text{spatial signal}$$

$$\text{spatial signal} \xrightarrow{\text{ReLU}} \text{spatial signal}$$

$$\text{spatial signal} \xrightarrow{\text{forward transform}} \text{coefficients.}$$

This has two important effects:

- The nonlinearity becomes well-defined (ReLU only ever touches real numbers).
- Information can now spread across different degrees l, which a purely per-degree linear operation could never do on its own.

As with regularization in other settings, this detour through the spatial domain is what makes the harmonic-domain network expressive enough to be useful, rather than just a fixed linear filter bank.

Rotation-Invariant Classification

For classification, we don't want the network's answer to depend on how the input happened to be rotated on the sphere, as a digit rotated to a different orientation is still the same digit.

Since rotation only ever changes a coefficient's phase ($e^{im\alpha}$) and never its magnitude, taking the magnitude $|a_l^m|$ of the final layer's output discards all memory of the rotation while keeping everything else. This gives a rotation-invariant feature vector, which is then passed through an ordinary fully-connected classifier.

This means equivariance is preserved through every convolution and nonlinearity, and is only discarded once, deliberately, at the very end.

Applying this to Digit Classification

Each digit image is projected onto a small patch of the sphere (as a "sticker" pinned to one region, not wrapped around the whole globe), converted into harmonic coefficients, and passed through several rounds of rotation-equivariant convolution and spatial-domain nonlinearity. Since a rotated coefficient set is exact rather than approximate, random rotations can be applied for free during training, teaching the network to be robust to how the digit sits on the sphere without any expensive data augmentation.

Practical Issues

1. **Information loss**: representing a signal with only finitely many harmonics (up to some maximum degree $l_{\max}$) necessarily discards fine detail. This can be reduced by increasing $l_{\max}$ and the density of the sampling grid, at the cost of more computation.

2. **Computational cost**: the convolution requires a separate weight matrix per degree, and the nonlinearity requires two full transforms (forward and inverse) per layer. To keep this manageable:

- the signal is confined to a small patch of the sphere rather than the whole surface,
- a moderate $l_{\max}$ and grid resolution are chosen as a deliberate trade-off between fidelity and speed,
- the transform matrices are precomputed once and reused across the entire dataset and every training step.

3. **Accuracy**: the highest accuracy achieved on sklearn's digit dataset was 97.5%, which is a mediocre performance. A standard CNN would go above 99%. Due to the experimental nature of the math involved in this project, as well as methods to reduce computation, this is acceptable.